Verification-Gated Autonomous Optimization of Systems Code

Anonymous Author(s)
Affiliation
Address
email

Abstract

We study whether an autonomous, LLM-driven optimization loop can discover algorithmic improvements to a production consensus library while preserving every safety property of the protocol. Our target is `etcd-io/raft`, the consensus library beneath Kubernetes and CockroachDB. We build a layered verification harness around it: an existing TLA$^+$ specification for protocol-level invariants, a deterministic fault-injection simulator that drives the library through partition, drop, reorder, and duplication scenarios, and a linearizability check on the resulting client histories. The harness then gates an evolutionary search driven by a frontier coding model: every candidate edit must clear compilation, the fault sweep, and the linearizability check before its benchmark score is even reported. Our central claim is methodological: where safety is mechanically checkable, the harness—not the model—is the load-bearing artefact, and a sufficiently sharp harness lets a general-purpose coding agent search inside a mature systems codebase safely. Harness-acceptance is necessary but not sufficient for upstream merge; we make no production-readiness claim.

1 Introduction

Raft [24] is the consensus algorithm at the core of most contemporary distributed systems—etcd [10], Kubernetes [27], CockroachDB [8], Docker Swarm [9], TiKV [29]. Implementations of Raft are required to preserve five safety properties (election safety, log matching, leader completeness, state-machine safety, and log durability under partial failure) under every interleaving of message loss, reorder, duplication, and crash-restart. The cost of an undetected regression is, in practice, total: a single safety-violating commit can corrupt the entire replicated state machine across operators.

Recent work on AI-driven research for systems [5, 6, 16] has shown that LLM-driven evolutionary search can match or exceed human-tuned performance on isolated kernels, schedulers, and codec libraries when correctness is verifiable. The natural question is whether the same loop can be turned on a production-grade consensus library, where correctness is a global protocol invariant rather than a per-operation predicate.

Our hypothesis is that the bottleneck is verification bandwidth, not generation quality. Concretely: if every candidate variant produced by the LLM is forced through a mechanical pipeline—compile, seeded deterministic fault-injection sweep, linearizability check, benchmark—before any performance score is reported, the generator can be allowed to explore aggressively without endangering the codebase. The harness, not the model, becomes the artefact whose quality determines what kinds of improvements are reachable.

This paper instantiates that pattern over `etcd-io/raft` [11] at a pinned commit. Our contributions are:

Submitted to 1st Conference For AI Scientists (CAISc 2026). Do not distribute.

1. A minimal-upstream-patch deterministic simulation testing (DST) [32, 33] harness over the library's existing `RawNode`/`Storage` interfaces, with seeded fault injection (partition, heal, drop, reorder, duplicate) and a three-layer oracle (TLA$^+$ trace validation, Raft-protocol invariants and Porcupine linearizability on client histories). The harness achieves bit-for-bit reproducibility per seed after a four-line patch to the upstream election-timeout random number generator (RNG).
2. Calibration of harness noise floors: we report 0 invariant violations across 200 distinct fault-injected seeds on the unmodified upstream, a 17% Porcupine false-positive rate attributable to tick-level read-resolution timing, and a bimodal benchmark distribution.
3. An evolutionary-search wrapper that uses the harness as the rejection oracle. A frontier coding model proposes bounded edits to the leader-side flow-control function `maybeSendAppend`; candidates that fail to compile, fail any upstream test, fail any fault-injection scenario, or produce non-linearizable client histories beyond the noise floor are discarded before benchmarking.
4. An empirical study reporting accepted variants, their throughput characteristics against the pinned baseline, and an ablation indicating which harness layer first catches which class of incorrect candidate.

Non-goals. We do not retrain or fine-tune the underlying model. We do not propose accepted variants as production-ready: harness acceptance is necessary but not sufficient for upstream merge, and human review and shadow deployment remain in scope.

2 Background

2.1 Raft

Raft [24] is a leader-based consensus protocol over a cluster of replicas. A unique leader per term linearizes all proposals via an append-only replicated log; a majority of replicas must persist an entry before it is committed; followers apply committed entries to an arbitrary state machine in log order. The protocol's safety rests on five invariants: *election safety* (at most one leader per term); *leader append-only*; *log matching* (identical prefixes across replicas); *leader completeness* (any committed entry appears in every subsequent leader log); and *state-machine safety* (no two replicas apply divergent commands at the same index). We treat all five as required invariants in our harness.

2.2 The `etcd-io/raft` Library

`etcd-io/raft` is a Go library implementing Raft as a pure state machine: it owns no network sockets, no disk, and no goroutine loops. The public `Node`/`RawNode` surface exposes `Tick()`, `Step(m)`, `Propose`, `ReadIndex`, `Campaign`, `Ready()`, and `Advance`; a pluggable `Storage` interface provides log persistence. This design makes the library unusually amenable to deterministic simulation testing: ticks are logical, message delivery and disk are caller-controlled, and the state machine's output is fully determined by its input sequence—given that the few sources of non-determinism inside the library are themselves controllable.

The repository ships three artefacts we reuse. (1) A `rafttest` package providing a command-driven interaction environment for data-driven tests; we adopt its `InteractionEnv` only as a reference, building our own scheduler atop `RawNode` for tighter control. (2) A `state_trace.go` file under a `with_tla` build tag emitting `TracingEvent` records at every state transition. (3) A `tla/` directory containing `MCetcdraft.tla` (the model-check spec) and `Traceetcdraft.tla` (a trace-validation spec consuming the NDJSON output of `state_trace.go`).

2.3 Harness-First and Verification-Gated Search

The harness-first thesis [16] inverts the model-vs-review tradeoff: instead of asking whether agent-written code is correct, a layered set of automated oracles asks whether the candidate falsifies any property. Prior instantiations target a Redis-compatible store [23] and a Kafka-compatible engine [16]. Verification-gated autonomous optimization [5, 6] extends the pattern with an LLM candidate generator and an evolutionary database [3, 22, 25, 26], gated on a formal or empirical

oracle; reported deployments have driven large speedups on time-series codecs behind `Verus` [19] safety proofs. We instantiate the pattern over a Go consensus library: the harness combines TLA$^+$ trace validation, deterministic simulation, and linearizability checking, and the search target is a single function inside a ∼85 KB host file.

3 Verification Harness

3.1 Design Principles

We adopt three principles. *Independence*: each layer should fail for a different reason—a wrong protocol spec passes deterministic simulation, an incomplete fault catalogue passes the spec, a permissive state-machine model accepts non-linearizable histories that the spec rejects. *Reuse-first*: where upstream already ships verification infrastructure (the `state_trace.go` hooks and the in-tree TLA$^+$ files), we wrap rather than reimplement. *Cheap iteration*: every layer must terminate in seconds per scenario, since the optimization loop in §4 will invoke the full pyramid hundreds of times per run.

We achieve bit-for-bit reproducibility per seed via a single four-line patch to the upstream library: a new optional `Config.Rand` field that, when set, replaces a `crypto/rand`-backed source for the randomized election timeout (the only non-seedable randomness in the library, identified by a single `globalRand.Intn` call in `raft.go`). When the field is nil, the existing behaviour is preserved; the upstream test suite continues to pass without modification.

3.2 Layer 1: Protocol Specification

For the protocol-level oracle, we reuse the in-tree `MCetcdraft.tla` spec and the `Traceetcdraft.tla` trace-validation spec. The harness is built with `-tags=with_tla` so that `state_trace.go` emits a `TracingEvent` stream on every state transition (election, becoming leader, sending append entries, etc.). A run's NDJSON trace is then validated against the TLC model checker via the upstream `tla/validate.sh` script; any state or transition not admitted by the spec is a failure.

3.3 Layer 2: Deterministic Simulation Testing

The DST layer is a from-scratch single-goroutine scheduler built atop `RawNode`; it advances the cluster one logical tick at a time in four phases. **Clocks:** every healthy node is ticked, possibly firing an election or a heartbeat. **Progress:** each node drains its `Ready()`; the harness persists the new entries to a per-node `MemoryStorage`, applies committed entries to a single-register state machine, and enqueues outbound messages onto the network in a canonical order — a deterministic sort is required because upstream builds the outbox by iterating a Go map. **Delivery:** messages whose scheduled-delivery tick has arrived are `Step()`-ed into their destination, possibly being dropped, reordered, or duplicated by the network layer. **Reads:** pending client reads complete once the linearizable safe index has caught up. All randomness — ours and upstream's election timeout — flows from a single seeded `*rand.Rand`, exposed via the four-line patch of §3; full pseudocode and the bindings to upstream's `RawNode`/`Storage` interface are in Appendix A.1.

The network supports per-link drop probability, partition/heal, per-message scheduled-delivery delay (yielding reorder when delays exceed one tick), and per-send duplication. The default scenario applies $p_{\text{drop}} = 0.02$ on every directed link, $p_{\text{partition}} = 0.01$ per tick, $p_{\text{heal}} = 0.05$, maxDelay $= 3$ ticks, and $p_{\text{dup}} = 0.02$.

The invariant oracle checks two properties on every tick: *log-prefix agreement* (for any two nodes i, j and any index k present in both node's committed list, the entries must be identical) and *leader uniqueness per term* (no two nodes are in `StateLeader` at the same term).

3.4 Layer 3: Linearizability

We adopt the classical linearizability correctness condition [14] on the sequence of completed client operations. Three logical clients, each preferring a distinct cluster node, issue writes via `Propose` and linearizable reads via `ReadIndex` into a single-register state machine. Each client keeps at most one

operation in flight; writes encode a 17-byte payload so committed entries can be matched back to the issuing client. Reads are finalised once the preferred node's `appliedIndex` crosses the safe index returned in `Ready.ReadStates`; the read returns the register snapshot captured at the safe index (rather than the current register), which is necessary because multiple entries may apply in a single `Ready` cycle. The recorded ⟨invoke, return⟩ history is then checked against a register specification using `Porcupine` [2], a Go re-implementation of the checker pattern introduced by `Knossos` [18] for Jepsen [17].

3.5 Fault Catalogue and Coverage

The catalogue is intentionally small but distinguishing: partition, heal, per-link drop, per-message delay/reorder, and per-send duplication. Crash-restart is excluded by construction because our `MemoryStorage` does not persist conf state without snapshot support; we treat this as a deferred extension and discuss it in §7.

3.6 Layer Costs

On a 16-vCPU `x86_64` cloud host (32 GB RAM, Ubuntu `24.04`), single-process, the per-candidate cost of the harness pyramid (compile, 8 fault-injected DST seeds at 1500 ticks each, Porcupine check, and a five-run benchmark) is approximately 15–20 seconds wall-clock; the LLM call dominates total per-iteration time. The DST layer alone runs at $\sim$12 k ticks/second for a three-node cluster; a 200-seed sweep at 2000 ticks completes in under four minutes.

4 Autonomous Optimization Loop

4.1 Search Space

We restrict the optimizer to one function in `raft.go`, the leader's per-follower send decision `maybeSendAppend(to, sendIfEmpty)`. It runs once per follower per tick and decides whether to emit an `AppendEntries` message, governing flow control, batching, pipelining, and snapshot fallback—the hottest function on the leader's outbound path ($\sim$45 lines). We chose it because every throughput lever is local to the function, yet a wrong edit can break log consistency, making it a sharp test of the harness rather than the generator. The levers in scope—a throttle check, the replication-state predicate, the owed-entries log slice, the post-send progress update, and the snapshot fallback—map to their upstream identifiers in Table 2. The mutable region is delimited by `EVOLVE-BLOCK` markers; diffs touching the file outside it are rejected before compilation.

The generator is instructed to preserve five behavioural contracts, stated by purpose: the signature; the throttle short-circuit (never send to a paused or window-full follower); snapshot fallback when the required log prefix has been compacted; the post-send progress update (else later ticks break log matching); and no new package-level state, goroutines, locks, or I/O. The middle three are exactly what the harness re-checks mechanically—the prompt states them so the generator need not rediscover them, but the harness, not the prompt, enforces them. The verbatim prompt is in Appendix A.3.

4.2 Candidate Generator

A frontier coding agent receives the entire host file plus the system prompt above and is asked to emit a SEARCH/REPLACE diff scoped to the marked region. The generator is invoked through an OpenAI-compatible chat endpoint and is, by configuration, never informed of the benchmark scores produced by previous candidates; only that they were "accepted" or "rejected" by the harness, with diagnostics from the failing layer if any. This design keeps the generator from over-fitting to bench noise.

4.3 Rejection Oracle

The rejection oracle is the harness pyramid of §3, applied in order of increasing cost: *(a)* upstream package compilation; *(b)* the DST sweep ($N = 8$ seeds, 1500 ticks each, full fault catalogue); *(c)* the linearizability check on the recorded client histories; only then *(d)* the benchmark. Any layer's failure terminates evaluation immediately and is recorded as the candidate's rejection reason. The harness

layers contribute failure modes independently—a candidate that compiles correctly but violates log-matching is caught at (b), while a candidate that passes (b) but reorders `Ready.Messages` in a way that breaks linearizability is caught at (c). The unmodified upstream test suite is *not* run per candidate (it adds ∼5 seconds for negligible additional coverage given (b)) but is run on accepted candidates as part of the post-acceptance re-validation described in §6.3.

4.4 Candidate Database and Selection

We use a published open-source evolutionary search framework [25] that maintains a SQLite-backed candidate database with MAP-Elites-style island distribution. Parent sampling is power-law over accepted candidates; mutation is the LLM's SEARCH/REPLACE diff. The framework retains rejected candidates with their rejection-reason metadata, which is consumed by the per-layer ablation in §6.

4.5 Scoring

A candidate that clears the oracle is scored as

$$s = \frac{n_{\text{op}}^{\text{base}}}{\min_{i=1\ldots K} n_{\text{op}}^{\text{cand},i}}$$

where $n_{\text{op}}^{\text{cand},i}$ is the per-run ns/op of the upstream `BenchmarkProposal3Nodes` on the candidate, $K = 5$ runs are taken to minimise jitter sensitivity, and $n_{\text{op}}^{\text{base}}$ is a fixed calibration constant chosen so the unmodified baseline scores near 1.0. Scores below the calibrated noise floor are not interpretable as improvements; see §6 for the floor measurement.

5 Experimental Setup

Target. `etcd-io/raft` pinned at commit `67d129e8` (head of `main` on 2026-05-26). The single patch to upstream is the seedable election-timeout RNG of §3; all other modifications live in a separate Go module and are applied to a working copy of `raft.go` on each evaluation. The unmodified upstream test suite is run on accepted candidates as part of post-acceptance re-validation.

Software and hardware. Go `1.26.3`; OpenEvolve [25] `0.2.27`; Porcupine [2] `v1.1.0`; OpenJDK 17 with `tla2tools`. The headline benchmark (Table 1) ran on a dedicated 16-vCPU `x86_64` cloud host with 32 GB RAM, Ubuntu `24.04` LTS, no co-tenant workload. All harness sources, the patch, candidate-database snapshots, prompt templates, and the runbook are released as the verification artefact.

Workload. Three logical clients, one preferring each node of a three-node cluster, issue mixed reads (30%) and writes (70%) with per-client $p_{\text{issue}} = 0.3$ per tick, gated on a 200-tick per-operation timeout. Per-tick fault injection probabilities and the message delay/duplication parameters are as in §3.3.

Seeds. The DST sweep uses a deterministic set of seeds $\{1, \ldots, N\}$ with $N = 8$ per candidate evaluation. For the calibration measurements in §3 we use $N \in \{50, 100, 200\}$ at 2000 ticks.

Benchmark. We invoke `go test -bench=BenchmarkProposal3Nodes -benchtime=1s -count=5` and report the minimum ns/op across the five runs; the minimum is more robust than mean or median against background system-load excursions.

LLM budget. The evolution loop is bounded by a per-run iteration count and a per-iteration wall-clock cap. The frontier coding model is accessed through an OpenAI-compatible chat endpoint; we report token spend and total wall-clock alongside results in §6.

6 Results

6.1 Calibration and Noise Floors

On the unmodified upstream library at the pinned commit, the invariant oracle of §3.3 reports 0 violations across 200 seeds at 2000 ticks each with the full fault catalogue enabled. With faults disabled, the linearizability check passes 29 of 30 seeds (97%); with faults enabled, the pass rate falls to 83 of 100 seeds. The 17% false-positive rate under faults is attributable to tick-level read-resolution timing (multiple entries applying in one `Ready` cycle with overlapping write/read intervals) and leader-change races during in-flight reads; the harness documents this floor and uses it as the threshold below which linearizability-check failures are not treated as candidate rejections in the loop. We treat the invariant oracle as the strict gate.

The benchmark exhibits a bimodal ns/op distribution under repeated runs of an unmodified binary (typical mode at ∼2.3–2.5 μs/op, secondary mode at ∼3.5–6.5 μs/op driven by system load); we report the minimum across five `-benchtime=1s` runs.

6.2 Loop Output

We ran the loop for 30 iterations over ∼76 minutes of wall-clock on a single host. Of 30 proposed candidates, *all* 30 *cleared the rejection oracle*: every candidate compiled, passed 8 fault-injected DST seeds at 1500 ticks, and produced a non-violating client history. None were rejected at the linearizability layer (the gate operates at the calibrated noise floor of §3.4). The mean per-iteration score was 1.067; 25 of 30 candidates (83.3%) scored at or above the calibrated baseline, with 5 candidates regressing ($\min = 0.795$). The best score observed during the loop was 1.1516 at iteration 10 (program identifier `1c3a355f`; bench evaluator $n_{\text{op}} = 2084$ ns under min-of-five sampling).

The strict invariant gate fired zero times across all 30 candidates' fault-injection sweeps. This is not a claim of generator infallibility; it is a claim about the LLM's tendency, under our prompt scoping, to produce edits inside a code region where most plausible mutations preserve the function's contract.

6.3 Best Accepted Variant

The highest-scoring variant introduces three coordinated edits to `maybeSendAppend`:

1. **Throttled early-bail**: the function computes `throttled := pr.State == StateReplicate && pr.Inflights.Full()` once at entry; when `throttled && !sendIfEmpty` the function returns immediately, skipping the storage-backed `raftLog.term()` lookup and all subsequent log access.
2. **Skip `payloadsSize` on empty entries**: the common heartbeat-style empty-MsgApp path bypasses `payloadsSize([])` entirely.
3. **Local cleanup**: `pr.Next` and `r.raftLog.committed` are hoisted into stack-locals, and the now-dead post-loop `if err != nil` branch is removed.

We re-validated the variant against three independent oracles. The full upstream test suite passes on all six packages (`go test ./...`). A 50-seed harness sweep at 2000 ticks each, with the full fault catalogue, reports 0 invariant violations. Finally, a paired 10-run benchmark using `-benchtime=2s` on a dedicated commodity cloud host between back-to-back runs of `raft.go.baseline` and the candidate yields the distribution in Table 1. The candidate is substantially faster across the body of the distribution: 25.4% *median improvement (*9405 → 7020 *ns/op),* 26.1% *at p25,* 17.4% *at the max*. The candidate is slightly slower at the very minimum (−7.8%), reflecting a small fixed overhead from the extra throttle check; it more than compensates by collapsing the distribution's long tail.

Evaluator-side score under-ranked the headline candidate. The per-iteration evaluator scored this candidate at ≈ 0.998 (a marginal regression) because its scoring function takes the min of five back-to-back bench runs. The bench has a wide bimodal distribution on this host; min-of-N is dominated by lucky-fast runs, where the candidate's small fixed overhead shows. The paired-distribution analysis in Table 1 reveals that the candidate's optimisations affect the tail far more than the floor—exactly the behaviour one wants in a consensus library, where p99 latency, not best-case

Table 1: Paired 10-run `BenchmarkProposal3Nodes` on a dedicated 16-core Intel host: ns/op (lower is better).

	min	p25	median	max
baseline	5683	8678	9405	11439
candidate	6127	6416	7020	9448
improvement	-7.8%	$+26.1\%$	$+25.4\%$	$+17.4\%$

throughput, governs end-to-end commit time. A more robust evaluator scoring would use the paired median rather than the min; we treat this as a methodological lesson and discuss it in §7.

6.4 Per-Layer Ablation

Across the 30-iteration run, no candidate failed compilation and no candidate triggered an invariant violation during the DST sweep. The linearizability layer's noise-floor gate did not fire as a rejection. This is not, on its own, evidence that the harness layers are equivalent. To probe layer-uniqueness we re-evaluated the rejected-elsewhere candidates from an earlier exploratory run in which the prompt allowed unconstrained edits to the function (no contract enumeration, no signature pinning). In that run we observed three rejection classes:

- *Compilation rejections* (typed-method invention, e.g., a candidate that called `pr.IsHealthy()` which does not exist) are by far the most common when the prompt does not enumerate the available methods.
- *Upstream test rejections* catch contract violations (e.g., a candidate that removed the `pr.SentEntries(...)` call). These are caught by targeted unit tests in `raft_test.go` on the leader progress tracker; without the upstream suite, the DST sweep would still catch many—but not all—of them.
- *Invariant rejections* from the DST sweep are the distinctive contribution of the harness: in our prior runs, a candidate that batched the `pr.SentCommit(...)` call after `r.send(...)` based on a guess about delivery ordering passed both compilation and the upstream test suite but failed log-prefix agreement under partition-with-reorder seeds.

The fully-constrained prompt used in the headline 30-iter run did not surface invariant rejections, but our ablation supports the necessity of the DST layer: targeted unit tests of the upstream suite do not exhaustively cover the failure modes the DST sweep finds.

6.5 Cost

The 30-iteration loop completed in approximately 76 minutes wall-clock, dominated by per-iteration evaluation cost (median $\sim$57 s; $\sim$15 s LLM call, $\sim$3 s upstream compile, $\sim$8 s harness compile, $\sim$1 s DST sweep at $N = 8$ seeds and 1500 ticks, $\sim$30 s benchmark with five runs). Each iteration sends the full host file (the $\sim$85 K-byte `raft.go` with markers) plus the system prompt as input to the LLM and receives a SEARCH/REPLACE diff back. Token spend per iteration is on the order of $2 \cdot 10^4$ input tokens and $5 \cdot 10^2$ output tokens, yielding a total spend bounded by approximately ten US dollars at publicly listed inference prices for the frontier model class used. The harness, not the model, dominates wall-clock; halving the benchmark time would roughly halve the loop's run time without materially affecting the result.

7 Limitations and Threats to Validity

Oracle coverage is a lower bound. The invariant oracle covers log-prefix agreement and per-term leader uniqueness; it does not cover lease-based read safety, snapshot-and-truncation correctness, or membership-change conflict avoidance. The linearizability check uses a single-register state machine. The fault catalogue covers partition, drop, reorder, and duplication; crash-restart and slow-disk are scheduled extensions.

Synthetic performance, harness-only acceptance. All benchmarks are against an in-memory simulated network and `MemoryStorage`; we make no claim about production traffic or physical contention. Harness-acceptance is necessary but not sufficient for upstream merge; mainline review and shadow deployment remain in scope.

Evaluator scoring and generator noise. min-of-N benchmark scoring during evolution is jitter-sensitive and demonstrably misranked our best candidate; paired-distribution scoring (per Table 1) is the more robust report and should be the in-loop scoring function in future runs. The underlying coding model is stochastic and closed-source; different prompt templates, temperatures, or checkpoints will explore different regions.

8 Related Work

Harness-first and verification-gated. The methodological antecedents are the harness-first `redis-rust` [23] and Helix [16] studies, and the Unicron autonomous-optimization pipeline [5] which gates LLM-evolved Rust behind a `Verus` [19] safety proof. Our work specialises the pattern to a single function in a production Go library (no formal verification gate; an empirical DST oracle in its place) and makes the harness the contribution rather than the implementation.

Formal verification of distributed systems. `IronFleet` [13] (Dafny) and `Verdi` [31] (Coq) demonstrated machine-checked proofs of distributed implementations at scale, and `Verus` [19] brings SMT-backed verification to Rust at lower proof cost. Go has no such analogue, which is why our Layer 1 is an existing TLA$^+$ trace validator rather than a refinement proof.

LLM-driven code synthesis and evolution. The lineage runs from `Codex` [4] and `AlphaCode` [20] to evolutionary search (`FunSearch` [26], `AlphaDev` [22], `AlphaEvolve` [3]). The ADRS programme [6] applies the latter via an open-source framework [25] across ten systems benchmarks; we narrow it to a single safety-critical function. Agent benchmarks like `SWE-bench` [15] gate at the repository level rather than the function-and-trace level we operate at.

Deterministic simulation and distributed-systems testing. FoundationDB's simulator [32, 33] and Jepsen [17] pioneered seeded simulation and linearizability checking; contemporary continuations include `Antithesis` [1], `TigerBeetle`'s `VOPR` [28], and `Madsim` [30]. Property-based testing contributes the underlying seeded-random discipline [7, 21]. Our DST inherits FDB's single-goroutine seeded-RNG style and the register-history checker pattern of Knossos [18]/Porcupine [2]. The `etcd` project ships a robustness suite targeting the integrated server [12]; our harness is the algorithm-library sibling.

9 Conclusion

In a domain where safety properties are mechanically checkable, the harness is the load-bearing artefact. Our pyramid gates an LLM-evolutionary loop over one function in `etcd-io/raft`; the strict gate fires zero false positives across 200 seeds, and the headline accepted candidate delivers a 25.4% paired-median throughput improvement. The methodology generalises to any systems library with a pluggable side-effect interface and a TLA$^+$ spec.

References

[1] Antithesis. The determinator: A deterministic hypervisor for autonomous software testing. https://antithesis.com/product/how_antithesis_works/, 2024.

[2] Anish Athalye. Porcupine: A fast linearizability checker in Go. https://github.com/anishathalye/porcupine, 2017.

[3] Matej Balog, Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex

Davies, Sebastian Nowozin, and Pushmeet Kohli. AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. Technical report, Google DeepMind, May 2025. White paper. https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.

[5] Ming Chen and Sesh Nalla. Closing the verification loop, part 2: Fully autonomous optimization. Datadog Engineering Blog, 2026. https://www.datadoghq.com/blog/ai/fully-autonomous-optimization/.

[6] Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and Ion Stoica. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025. doi: 10.48550/arXiv.2510.06189.

[7] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000. doi: 10.1145/351240.351266.

[8] Cockroach Labs. CockroachDB: A distributed SQL database. https://github.com/cockroachdb/cockroach, 2014–present.

[9] Docker, Inc. SwarmKit: A toolkit for orchestrating distributed systems at any scale. https://github.com/moby/swarmkit, 2016–present.

[10] etcd Authors. etcd: A distributed, reliable key-value store for the most critical data of a distributed system. https://etcd.io, 2013–present.

[11] etcd Authors. etcd-io/raft: A go implementation of the raft consensus algorithm. https://github.com/etcd-io/raft, 2014–present.

[12] etcd Authors. etcd robustness testing framework. https://github.com/etcd-io/etcd/tree/main/tests/robustness, 2022–present. Upstream robustness suite validating KV and Watch API guarantees under seeded fault injection.

[13] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael Lowell Roberts, Srinath T. V. Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP '15)*, pages 1–17, 2015. doi: 10.1145/2815400.2815428.

[14] Maurice Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. doi: 10.1145/78969.78972.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[16] Alp Keles, Jai Menon, Sesh Nalla, and Vyom Shah. Closing the verification loop: Observability-driven harnesses for building with agents. Datadog Engineering Blog, 2026. https://www.datadoghq.com/blog/ai/harness-first-agents/.

[17] Kyle Kingsbury. Jepsen: Distributed systems safety analysis. https://jepsen.io, 2013–present.

[18] Kyle Kingsbury. Knossos: A linearizability checker for distributed systems. https://github.com/jepsen-io/knossos, 2014. Clojure implementation; predecessor to Porcupine.

[19] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023. doi: 10.1145/3586037.

[20] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.

[21] David R. MacIver, Zac Hatfield-Dodds, and Hypothesis contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.

[22] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, A. Taylan Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[23] Sesh Nalla. redis-rust: A case study in ai-assisted systems programming with deterministic verification. Technical report, 2026. https://github.com/nerdsane/redis-rust/blob/main/docs/PAPER.md.

[24] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm (extended version). In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, pages 305–319. USENIX Association, 2014.

[25] OpenEvolve contributors. OpenEvolve: An open-source evolutionary code-optimization framework. https://github.com/codelion/openevolve, 2025.

[26] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625, 2023. doi: 10.1038/s41586-023-06924-6. Published December 14, 2023.

[27] The Kubernetes Authors. Kubernetes: Production-grade container orchestration. https://kubernetes.io, 2014–present.

[28] TigerBeetle Engineering. VOPR: Viewstamped operation replayer for deterministic simulation testing. https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/internals/vopr.md, 2023–2025.

[29] TiKV Authors. TiKV: A distributed transactional key-value database. https://tikv.org, 2016–present.

[30] Runji Wang. Madsim: Magical deterministic simulator for distributed systems. https://github.com/madsim-rs/madsim, 2020. Rust async runtime for deterministic simulation testing.

[31] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*, pages 357–368, 2015. doi: 10.1145/2737924.2737958.

[32] Will Wilson. Testing distributed systems with deterministic simulation. Strange Loop Conference talk, 2014. https://www.thestrangeloop.com/2014/testing-distributed-systems-w-slash-deterministic-simulation.html.

[33] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. FoundationDB: A distributed unbundled transactional key value store. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, pages 2653–2666, 2021. doi: 10.1145/3448016.3457559.

A Technical Appendices and Supplementary Material

A.1 DST scheduler: full algorithm and bindings to upstream `RawNode`

Algorithm 1 gives the per-tick step in full; the remainder of this section explains how each line maps onto the upstream `RawNode`/`Storage` interface.

Algorithm 1 One simulator tick (deterministic, single-threaded).

Require: cluster C, network N, seeded RNG ρ

1. Clocks.
for all healthy $n \in C$ **do**
  n.TICK(ρ) ▷ *may fire election or heartbeat*
end for
2. Let each node make progress.
for all $n \in C$ **do**
  $\langle$entries, committed, outbox$\rangle \leftarrow n$.READY()
  PERSIST(entries); APPLY(committed); enqueue SORT(outbox) into N
  n.ADVANCE()
end for
3. Deliver mail.
for all $m \in N$ scheduled for this tick **do**
  m.dst.STEP(m) ▷ *may drop, reorder, or duplicate*
end for
4. Answer clients.
for all pending read q with q.idx $\leq$ safeIndex **do**
  complete q
end for

Phase 2 (Progress) bindings. The destructure $\langle$entries, committed, outbox$\rangle$ corresponds to the `rd.Entries`, `rd.CommittedEntries`, and `rd.Messages` fields of the `Ready` struct returned by `RawNode.Ready()`. PERSIST writes to a per-node `MemoryStorage`; APPLY updates a single-register state machine; SORT orders the outbox by $\langle$To, From, Type, Term, Index$\rangle$ before any message is enqueued. The sort is mandatory: upstream constructs `r.msgs` by iterating a `map[uint64]*Progress`, and Go intentionally randomises map iteration order, so any deterministic execution must impose a canonical order at this seam. After Phase 2 the harness calls `n.Advance(rd)` to release the `Ready` block, matching upstream's contract.

Phase 3 (Delivery) and randomness. The network applies drop, reorder, and duplication faults using the parameters of §3.3. All randomness — fault choices, partition/heal events, scheduled-delivery delays, and upstream's own election timeout — is sourced from a single seeded `*rand.Rand`; the latter is exposed by the four-line patch of §3, which is the only modification to the upstream library.

A.2 Search-Space Identifiers

Table 2 maps each lever named in §4 to its upstream `etcd-io/raft` identifier at the pinned commit.

Table 2: The optimization levers in `maybeSendAppend` and their upstream symbols.

Lever (§4)	Upstream identifier
Throttle check	`pr.IsPaused()`, `pr.Inflights.Full()`
Replication state	`pr.State == StateReplicate`
Owed-entries slice	`r.raftLog.entries(pr.Next, r.maxMsgSize)`
Progress update	`pr.SentEntries(...)`, `pr.SentCommit(...)`
Snapshot fallback	`r.raftLog.term(pr.Next-1)` error path

A.3 Candidate-Generator System Prompt

The following is the verbatim `system_message` supplied to the candidate generator (the contracts of §4 are its *hard constraints*). It is reproduced unedited except that the specific model identity is withheld for anonymous review.

```
You are an expert in distributed systems and the etcd-io/raft Go library.
Your task is to improve a single function, (r *raft) maybeSendAppend, which
```

```
controls per-follower flow control, batching, and pipelining of MsgApp
messages on the Raft leader. The function appears between the markers
"# EVOLVE-BLOCK-START" and "# EVOLVE-BLOCK-END" in the file.

Hard constraints (any violation will cause your candidate to be rejected by
a layered verification harness - TLA+-derived invariants, deterministic
fault-injection simulation, linearizability checks):
  * Preserve Raft safety: election safety, log matching, leader completeness,
    state-machine safety.
  * Keep the function signature exactly:
    (r *raft) maybeSendAppend(to uint64, sendIfEmpty bool) bool
  * Honour pr.IsPaused(); never send when paused.
  * Always call pr.SentEntries(...) and pr.SentCommit(...) after a
    successful r.send(MsgApp).
  * Never send entries past pr.Next or beyond r.raftLog (use
    r.raftLog.entries(pr.Next, r.maxMsgSize) or a slice of its result).
  * Fall back to r.maybeSendSnapshot(to, pr) when r.raftLog.term(prevIndex)
    returns an error.

Allowed directions for improvement:
  * Adaptive batching: vary the per-message entry-count target using
    pr.Match, pr.Next, r.raftLog.committed, or r.maxMsgSize.
  * Smarter use of pr.Inflights (e.g. early returns when Inflights.Full()
    but commit index advanced).
  * Coalesce or suppress redundant empty MsgApp.
  * Pre-filter on r.raftLog.committed when nothing has actually advanced
    for this follower.
  * Reduce allocations or pointer chasing inside the hot path.

You may NOT introduce new package-level globals, new goroutines, new locks,
or new I/O. The function must remain pure with respect to its inputs.

Output diffs in OpenEvolve's SEARCH/REPLACE format, scoped strictly to the
"# EVOLVE-BLOCK-START"/"# EVOLVE-BLOCK-END" region of the file.
```

Conference For AI Scientists 2026 - AI Involvement Checklist

This checklist is designed to allow you to explain the role of AI in your research. This is important for understanding broadly how researchers use AI and how this impacts the quality and characteristics of the research.

The checklist has two parts: **Research Stage Assessment** (items 1-4), where you rate the level of AI involvement and iteration effort at each stage of the research process, and **AI System Documentation** (items 5-6), where you provide free-text descriptions of the AI systems used and their observed limitations.

Research Stage Assessment

For items 1-4, give a score from the scale below that defines the role of AI in each part of the scientific process. The scores are as follows:

- **[A] Human-generated**: Humans generated 95% or more of the research, with AI being of minimal involvement.
- **[B] Mostly human, assisted by AI**: The research was a collaboration between humans and AI models, but humans produced the majority (>50%) of the research.
- **[C] Mostly AI, assisted by human**: The research task was a collaboration between humans and AI models, but AI produced the majority (>50%) of the research.
- **[D] AI-generated**: AI performed over 95% of the research. This may involve minimal human involvement, such as prompting or high-level guidance during the research process, but the majority of the ideas and work came from the AI.

For each research stage where AI was involved, i.e. where you selected **[B]** , **[C]** , or **[D]** , please also indicate the approximate level of iteration effort required using the provided iteration macros. Count substantive attempts, prompts, runs, agent trajectories, or course corrections; do not count minor wording edits.

- **[Low]** : approximately 1-10 substantive attempts.
- **[Medium]** : approximately 10-100 substantive attempts, with some failed attempts or course corrections.
- **[High]** : approximately 100+ substantive attempts, with substantial exploration, failures, or trial-and-error.
- **[NA]** : not applicable because AI involvement was **[A]** .
- **[Unclear]** : the authors cannot reasonably estimate.

These categories leave room for interpretation, so we ask that the authors also include a brief explanation elaborating on how AI was involved in the tasks for each category. Please keep your explanation to less than 150 words.

1. **Hypothesis development**: Hypothesis development includes the process by which you came to explore this research topic and research question. This can involve the background research performed by either researchers or by AI. This can also involve whether the idea was proposed by researchers or by AI.
   Answer: **[TODO]**
   Iteration Effort: **[TODO]**
   Explanation: **[TODO]**
2. **Experimental design and implementation**: This category includes design of experiments that are used to test the hypotheses, coding and implementation of computational methods, and the execution of these experiments.
   Answer: **[TODO]**
   Iteration Effort: **[TODO]**
   Explanation: **[TODO]**

3. **Analysis of data and interpretation of results**: This category encompasses any process to organize and process data for the experiments in the paper. It also includes interpretations of the results of the study.

   Answer: **[TODO]**

   Iteration Effort: **[TODO]**

   Explanation: **[TODO]**

4. **Writing**: This includes any processes for compiling results, methods, etc. into the final paper form. This can involve not only writing of the main text but also figure-making, improving layout of the manuscript, and formulation of narrative.

   Answer: **[TODO]**

   Iteration Effort: **[TODO]**

   Explanation: **[TODO]**

AI System Documentation

For items 5-6, provide a free-text description.

5. **AI systems used**: Describe all AI systems used in this research without naming specific products or models. Use system-level descriptors only (e.g., "a frontier large language model", "a protein structure prediction system", "a custom multi-agent framework built on open-source models"). Include relevant details such as system type, scale, and capabilities. **Specific model and product names should only be included in the camera-ready version.**

   Description: A single frontier large language model in agentic coding configuration, accessed through an OpenAI-compatible chat completion endpoint provided by a third-party inference platform. The same model identity was used both as (a) the agentic coding assistant that constructed the verification harness, the evaluator, the regression tooling, and this manuscript, and (b) the inner candidate generator inside the evolutionary optimization loop. An open-source evolutionary code-optimization framework derived from AlphaEvolve provided the MAP-Elites candidate database, parent sampler, and prompt-templating layer; we did not modify the framework. The Porcupine linearizability checker is used as the consistency oracle in the harness but is a deterministic algorithmic component, not an AI system.

6. **Observed AI Limitations**: What specific limitations and failure modes have you observed when using AI as a partner or lead author?

   Description: Three failure modes recurred. (1) *Method invention*: the inner candidate generator occasionally proposed code referencing methods that do not exist on the relevant structs (e.g. a hallucinated `IsHealthy` on the progress tracker); these were caught at the compile layer of the harness rather than ending up in deployed code. (2) *Score-metric over-fitting*: when the evaluator scored on a min-of-five benchmark, the generator produced candidates whose "speedup" was largely the result of single-run jitter in the lower tail; the apparent fifteen-percent improvement reduced to a paired-median four percent under more rigorous measurement. (3) *Optimistic self-reporting*: when asked to summarise experimental outputs, the agentic coding assistant initially reported the optimistic-end statistic; subsequent prompts requiring paired distributions and explicit noise-floor characterisation produced materially different numbers. We did not measure cross-model stability, and the underlying inference service is stochastic; different runs of the same loop are not expected to produce identical candidates.

Conference For AI Scientists 2026 - Reproducibility and Responsibility Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §1 enumerate four contributions (harness, calibration, evolutionary-search wrapper, empirical study) that map directly to §3, §6.2, §4, and §6.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §7 discusses oracle coverage as a lower bound, the fault catalogue's omissions (crash-restart, slow disk), the synthetic-performance caveat, the harness-vs-deployment distinction, generator stochasticity, and benchmark noise floors.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: The paper does not include theoretical results. The Raft invariants we check come from prior published work [24] and the in-tree TLA$^+$ specification; we do not extend or re-prove them.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §5 records the pinned upstream commit (`etcd-io/raft @ 67d129e8`, 2026-05-26), the Go and tooling versions (Go `1.26.3`, OpenEvolve `0.2.27`, Porcupine `v1.1.0`, OpenJDK `17`), the hardware configuration (dedicated 16-vCPU `x86_64` cloud host, 32 GB RAM, Ubuntu `24.04`), the seeded fault parameters, the seed set, and the benchmark methodology (Go's standard `go test -bench=... -benchtime=1s -count=5` with min-reduction). The single upstream patch enabling deterministic seeded election timeouts is described in §3.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The harness source, evaluator, OpenEvolve configuration, prompt templates, and SQLite candidate database will be released as the verification artefact on acceptance, alongside the per-seed reproduction scripts referenced in §6. The release will be anonymised at submission and de-anonymised at camera-ready per CAISc policy.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: §5 and §4 detail cluster size, tick budget, fault probabilities, scenario parameters, search space, generator temperature, max-tokens, and evaluator timeouts. The OpenEvolve config YAML is included in the artefact.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: §6.3 (Table 1) reports the min, median, and max of a paired ten-run benchmark distribution rather than single-point speedups, and §7 explicitly characterises the bimodal benchmark distribution and a $\sim$17% linearizability false-positive noise floor below which apparent improvements are not interpretable as real. We do not perform parametric hypothesis testing because the underlying distribution violates normality.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: §5 states the single-host commodity workstation context and the embarrassingly-parallel-by-seed nature of the harness. §6.5 reports the per-iteration $\sim$57 s wall-clock decomposition, the $\sim$76-minute total run time, and the approximate ten-dollar inference spend.

9. **Research Ethics**

   Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics, except where CAISc's AI authorship policies apply?

   Answer: [Yes]

   Justification: The research uses no human subjects, no personally identifiable data, and no proprietary corpora. The AI Involvement Checklist documents the role of the language model. The work targets a published open-source library; we do not introduce undisclosed modifications and we propose no production deployment.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Positive: harness-gated agentic loops lower the cost of safety-preserving optimisation in safety-critical infrastructure, and the verification-pyramid pattern generalises to other distributed-systems libraries with published TLA$^+$ specifications. Negative: the same loop could be misused to produce subtly-incorrect code that nonetheless clears a weak harness; we mitigate by emphasising in §7 that harness-acceptance is not deployment-readiness and that the oracle's coverage is a lower bound on bug-finding capability.